Zellic

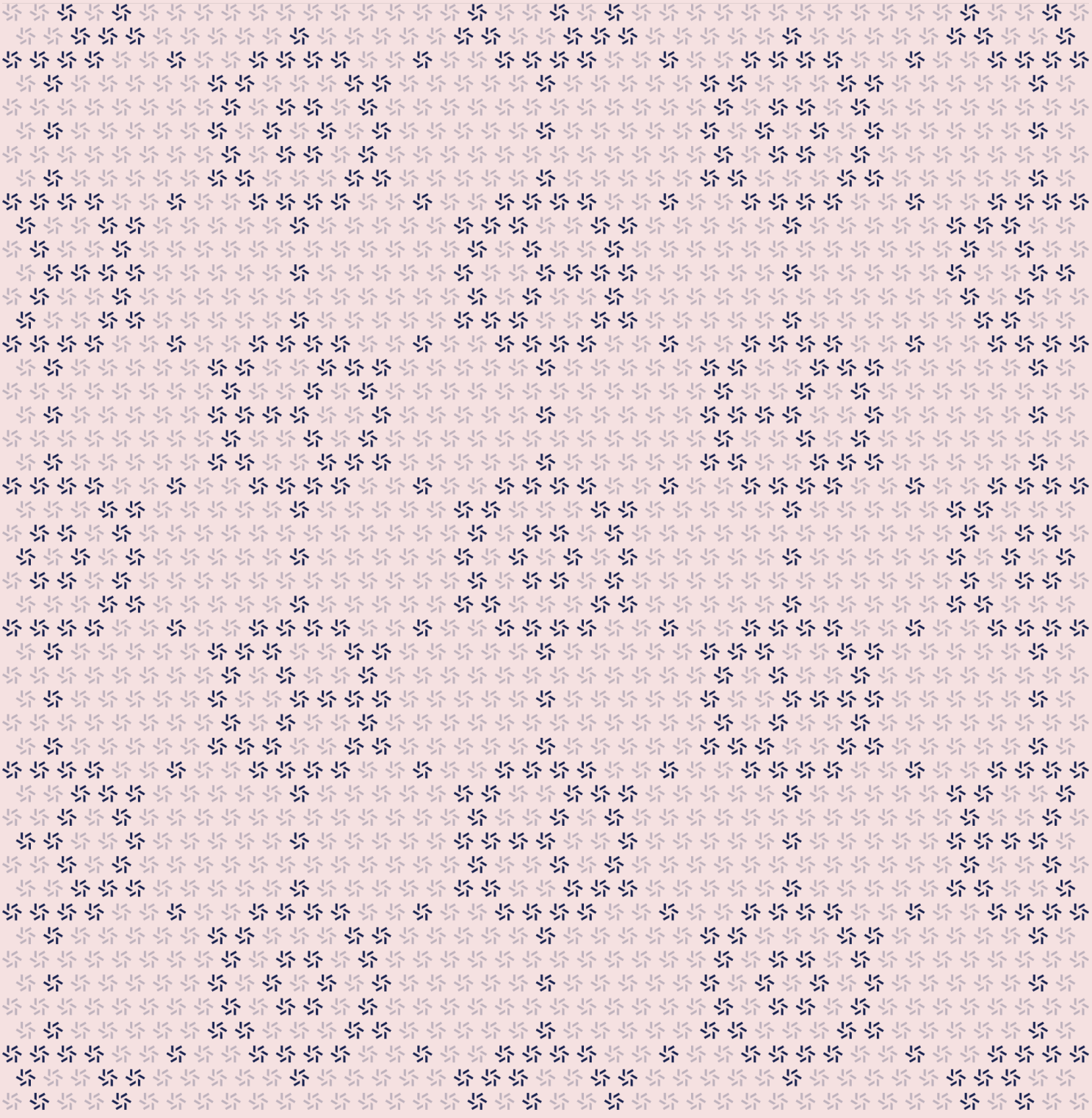
Prepared for
Kelly Zhang
LayerZero Labs

Prepared by
Ethan Lee
Ayaz Mammadov
Zellic

April 2, 2026

LayerZero Stellar OneSig

Stellar Application Security Assessment



Contents

About Zellic **4**

1. Overview **4**

1.1. Executive Summary 5

1.2. Goals of the Assessment 5

1.3. Non-goals and Limitations 5

1.4. Results 5

2. Introduction **6**

2.1. About LayerZero Stellar OneSig 7

2.2. Methodology 7

2.3. Scope 9

2.4. Project Overview 9

2.5. Project Timeline 11

3. Detailed Findings **12**

3.1. Outdated Stellar Soroban SDK version could result in issues with function wiring 13

3.2. Opaque leaf hashes make signer-as-executor approvals hard to verify 15

3.3. Cross Tree Replay Gap 18

4. Discussion **19**

4.1. Duplicate signer logic 20

4.2. Incorrect comment 20

4.3. Scope-change details 21

5. System Design **23**

6. Assessment Results **24**

6.1. Disclaimer 25

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the #1 CTF (competitive hacking) team ↗ worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io ↗ and follow @zellic_io ↗ on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io ↗.



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for LayerZero Labs from January 28th to February 26th, 2026. During this engagement, Zellic reviewed LayerZero Stellar OneSig's code for security vulnerabilities, design issues, and general weaknesses in security posture. This report is a subset of a broader report covering the Stellar implementation of the Layerzero Endpoint V2.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Are there any vulnerabilities that could allow for a single signer to execute a message?
- Are there any issues in the Merkle proof verification and authorization?

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Off-chain components
- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

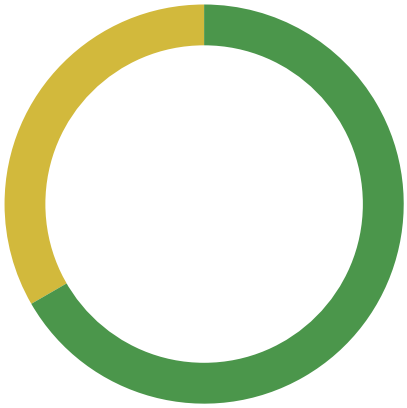
1.4. Results

During our assessment on the scoped LayerZero Stellar OneSig files, we discovered three findings. No critical issues were found. One finding was of medium impact and two were of low impact.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of LayerZero Labs in the Discussion section (4. ↗).

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	1
Low	2
Informational	0



2. Introduction

2.1. About LayerZero Stellar OneSig

LayerZero Labs contributed the following description of LayerZero Stellar OneSig:

> LayerZero works with asset issuers, chains, and applications to make money more extensible. Issue any asset on chain, extend its reach to new ecosystems, connect fragmented liquidity and more.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the files.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood.

We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped files itself. These observations — found in the Discussion (4. ↗) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets:

LayerZero Stellar OneSig Files

Type	Rust
Platform	Stellar-Soroban
Target	monorepo-internal
Repository	https://github.com/LayerZero-Labs/monorepo-internal ↗
Version	`8f68242a42bfa5556f04e68bc70cbab4c658fc88`
Programs	`packages/onesig/onesig-stellar/contracts/onesig`

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 5.7 person-weeks. The assessment was conducted by two consultants over the course of 4.4 calendar weeks.

Contact Information

The following project manager was associated with the engagement:

Jacob Goreski
Engagement Manager
jacob@zellic.io

The following consultants were engaged to conduct the assessment:

Ethan Lee
Engineer
ethl@zellic.io

Ayaz Mammadov
Engineer
ayaz@zellic.io

2.5. Project Timeline

The key dates of the engagement are detailed below.

January 28, 2026	Start of primary review period
February 12, 2026	Scope change from `8f68242a` to `b659ed83`
February 16, 2026	Scope change from `b659ed83` to `00fca66e`
February 26, 2026	End of primary review period
March 11, 2026	Start of secondary review period
March 11, 2026	Scope change from `00fca66e` to `2ed91ed3`
March 12, 2026	Scope change from `2ed91ed3` to `fe230189`
March 12, 2026	Scope change from `fe230189` to `cf2ddcfa`
March 14, 2026	End of secondary review period
March 23, 2026	Start of tertiary review period
March 23, 2026	Scope change from `cf2ddcfa` to `fe2f3954`
March 24, 2026	Scope change from `fe2f3954` to `3822cc60`
March 25, 2026	End of tertiary review period
March 26, 2026	Start of quaternary review period
March 27, 2026	Scope change from `3822cc60` to `3c78439e`
March 27, 2026	End of quaternary review period

April 29, 2026	Start of quinary review period
April 29, 2026	The repository was migrated to audit-external ↗
April 29, 2026	Scope change from 3c78439e ↗ to ebd661e3 ↗ in audit-external
April 30, 2026	End of quinary review period
May 7, 2026	Start of senary review period
May 7, 2026	Scope change from ebd661e3 ↗ to bf9ea5e4 ↗
May 14, 2026	End of senary review period
June 17, 2026	Start of septenary review period
June 17, 2026	Scope change from bf9ea5e4 ↗ to facaf223 ↗
June 17, 2026	End of septenary review period
June 23, 2026	Scope migrated to repository monorepo-external ↗ at an equivalent commit dcade8e
July 22, 2026	Start of octonary review period
July 22, 2026	Scope change from dcade8e7 ↗ to 0781f4e6 ↗
July 22, 2026	End of octonary review period

3. Detailed Findings

3.1. Outdated Stellar Soroban SDK version could result in issues with function wiring

Target	Project-wide		
Category	Business Logic	**Severity**	Medium
Likelihood	Medium	**Impact**	Medium

Description

At the time of writing, the Stellar Soroban version being used is V25.1.0. This version is affected by Stellar security advisory GHSA-4chv-4c6w-w254 ↗.

The `#[contractimpl]` macro contains a bug in how it wires up function calls.

In Rust, functions on a type can be defined in two ways:

1. Directly on the type as an inherent function

```
impl MyContract {
    fn value() { ... }
}
```

2. Through a trait

```
impl Trait for MyContract {
    fn value() { ... }
}
```

These are two separate functions that happen to share the same name. Rust has rules for which one is called. When writing `MyContract::value()`, Rust always picks the one defined directly on the type, not the trait version.

Impact

As there is a lot of macro usage as well as exposed macros, there is a risk that developers writing OApps, or developers working on LayerZero, encounter the edge case associated with the published vulnerability.

Recommendations

Upgrade to a more secure and up-to-date version of Stellar Soroban SDK.

Remediation

This issue has been acknowledged by LayerZero Labs, and a fix was implemented in PR #2812 ↗. They intend to merge these changes into the production branch.

3.2. Opaque leaf hashes make signer-as-executor approvals hard to verify

Target	Auth		
Category	Cryptography	**Severity**	Medium
Likelihood	Low	**Impact**	Low

Description

The signer-as-executor flow asks the Secp256k1 signer to authorize a `leaf_hash`, not the transaction contents in a form that the signer can inspect. The leaf hash is derived from the OneSig ID, contract address, nonce, and encoded calldata, then hashed twice.

The calldata contains the destination contract, function name, arguments, and nested subinvocations. These are the fields that define what the signer is authorizing.

```
// packages/onesig/onesig-
    stellar/contracts/onesig/src/interfaces/onesig.rs#L8-L20
pub struct Call {
    pub to: Address,
    pub func: Symbol,
    pub args: Vec<Val>,
    pub sub_invocations: Vec<InvokerContractAuthEntry>,
}
```

However, the signer-as-executor proof does not sign this `Call` struct directly. The contract first serializes it to XDR, mixes it with the OneSig metadata and nonce, and then hashes the result into a 32-byte leaf.

```
// packages/onesig/onesig-stellar/contracts/onesig/src/onesig.rs#L101-L117
fn encode_leaf(env: &Env, nonce: u64, call: &Call) -> BytesN<32> {
    let mut writer = BufferWriter::new(env);
    let leaf_data = writer
        .write_u8(LEAF_ENCODING_VERSION)
        .write_u64(Self::onesig_id(env))
        .write_address_payload(&env.current_contract_address())
        .write_u64(nonce)
        .write_bytes(&call.to_xdr(env))
        .to_bytes();

    env.crypto()
        .keccak256(&env.crypto().keccak256(&leaf_data).into())
```

```
        .into()
}
```

The signer-as-executor proof then commits to this opaque `leaf_hash`, the delegate, and the proof expiry.

```
// packages/onesig/onesig-stellar/contracts/onesig/src/eip712.rs#L77-L95
pub fn build_signer_proof_digest(
    env: &Env,
    leaf_hash: &BytesN<32>,
    delegate: &BytesN<32>,
    signer_proof_expiry: u64,
) -> BytesN<32> {
    let inner_bytes = BufferWriter::new(env)
        .write_bytes_n(leaf_hash)
        .write_bytes_n(delegate)
        .write_u64(signer_proof_expiry)
        .to_bytes();
    let inner_hash = env.crypto().keccak256(&inner_bytes);

    let digest_bytes = BufferWriter::new(env)
        .write_array(&EIP191_PERSONAL_SIGN_PREFIX_32)
        .write_array(&inner_hash.to_array())
        .to_bytes();
    env.crypto().keccak256(&digest_bytes).into()
}
```

This means the signer does not directly approve readable call details such as the destination contract, target function, arguments, subinvocations, and nonce. Those fields are only indirectly represented inside a hash. If the off-chain signing UI or coordinator presents incorrect transaction details, the signer has no way to verify that the displayed transaction matches the opaque leaf being signed.

Impact

An attacker still needs a valid threshold-signed Merkle root and matching Merkle proof, so this does not allow arbitrary transaction execution. However, a compromised or malicious coordinator can trick a signer into producing a signer-as-executor proof for a malicious leaf message containing malicious call details other than the ones shown in the UI.

Recommendations

Have signers approve structured signer-as-executor data that includes the decoded call details, delegate, nonce, target OneSig contract, and expiry. At minimum, signing tooling should

independently decode the leaf preimage and display the exact call fields before requesting the proof.

Remediation

This issue has been acknowledged by LayerZero Labs, and a fix was implemented in commit 8a1e2f70 ↗.

3.3. Cross Tree Replay Gap

Target	authz		
Category	Business Logic	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

```
/// Verifies the signer-as-executor flow per the spec.
fn verify_signer_as_executor_permissions(
    env: &Env,
    signature_payload: &BytesN<32>,
    leaf_hash: &BytesN<32>,
    proof: &SignerAsExecutor,
) {
    ...
    // 2. proving `delegate` actually signed the Soroban runtime payload.
    env.crypto().ed25519_verify(
        &proof.delegate,
        &signature_payload.clone().into(),
        &proof.delegate_proof,
    );

    // 3. The recovered secp256k1 address is a registered signer.
    let digest =
        build_signer_proof_digest(env, leaf_hash, &proof.delegate,
    proof.signer_proof_expiry);
    let signer = multisig::recover_signer(env, &digest, &proof.signer_proof);
    ...
}
```

If there are multiple active merkle roots, and the signer publishes a signer proof for the execution of a leaf, that proof could be used for an identical leaf in another active merkle tree. However, this is unlikely to happen due to the fact that leaves contain globally stored nonces and the collision of the hashes are improbable.

Impact

A published signer_proof could be used for another leaf in another merkle tree, allowing early execution of a onesig call.

Recommendations

Bind the merkle root to the signer proof to prevent the cross-tree replay.

Remediation

This issue has been acknowledged by LayerZero Labs, and a fix was implemented in commit bf9ea5e4 ↗.

4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

4.1. Duplicate signer logic

The `add_signer` function disallows adding the same keys to the multi-sig, though it seems that two different public keys can be generated from the same private key, in which case this check would not be effective. However, it seems that the purpose of this validation is to prevent accidental inclusion of the same signer twice and not to guarantee one-signer-to-one-user correspondence.

```
fn add_signer(env: &Env, signer: &BytesN<20>) {
    // Not allowed to add zero address as signer
    assert_with_error!(env, signer != &BytesN::from_array(env, &[0u8; 20]),
    MultiSigError::InvalidSigner);
    // Not allowed to add same signer twice
    let mut signers = MultiSigStorage::signers(env);
    assert_with_error!(env, !signers.iter().any(|s| &s == signer),
    MultiSigError::SignerAlreadyExists);

    // Add signer to list
    signers.push_back(signer.clone());
    MultiSigStorage::set_signers(env, &signers);

    SignerSet { signer: signer.clone(), active: true }.publish(env);
}
```

4.2. Incorrect comment

In the logic in eip.rs, the comment detailing the domain separator is incorrect.

```
/// Pre-computed EIP-712 domain separator for merkle-root signing and
/// keccak256(abi.encode(EIP712DOMAIN_TYPE_HASH, keccak256("OneSig"),
    keccak256("0.0.1"), 1, 0xdEaD))
const EIP712_DOMAIN_SEPARATOR: [u8; 32] = [
    0x94, 0xc2, 0x89, 0x89, 0x17, 0x0e, 0xb4, 0xdc, 0x31, 0x35, 0x91, 0x74,
    0xb9, 0x11, 0x5c, 0x11,
    0x6a, 0x8f, 0xaf, 0xa6, 0x7b, 0x5a, 0xda, 0xcc, 0x57, 0x0c, 0xa5, 0x83,
    0xeb, 0x96, 0xd6, 0x57,
```

```
];
```

The comment instead should state the following:

```
/// Pre-computed EIP-712 domain separator for merkle-root signing and signer
    execution authorization
```

4.3. Scope-change details

Here we outline the details of each scope change.

February 12th, 2026

A scope change occurred on February 12th, 2026. The change was from `8f68242a` to `b659ed83`. Zellic conducted a diff audit between the previous scope and the new scope.

February 16th, 2026

A scope change occurred on February 16th, 2026. The change was from `b659ed83` to `00fca66e`. Zellic conducted a diff audit between the previous scope and the new scope.

March 11th, 2026

A scope change occurred on March 11th, 2026. The change was from `00fca66e` to `2ed91ed3`. Zellic conducted a diff audit between the previous scope and the new scope.

March 12th, 2026

Two scope changes occurred on March 12th, 2026.

The first change was from `2ed91ed3` to `fe230189`. Zellic conducted a diff audit between the previous scope and the new scope.

The second change was from `fe230189` to `cf2ddcfa`. Zellic conducted a diff audit between the previous scope and the new scope.

March 23rd, 2026

A scope change occurred on March 23rd, 2026. The change was from cf2ddcfa to fe2f3954. Zellic conducted a diff audit between the previous scope and the new scope.

March 24th, 2026

A scope change occurred on March 24th, 2026. The change was from fe2f3954 to 3822cc60. Zellic conducted a diff audit between the previous scope and the new scope.

March 27th, 2026

A scope change occurred on March 27th, 2026. The change was from 3822cc60 to 3c78439e. Zellic conducted a diff audit between the previous scope and the new scope.

April 29th, 2026

A scope change occurred on April 29th, 2026. The repository was migrated to audit-external. The change was from 3c78439e in LayerZero-Labs/monorepo-internal to ebd661e3 in LayerZero-Labs/audit-external. Zellic conducted a diff audit between the previous scope and the new scope.

May 7th, 2026

A scope change occurred on May 7th, 2026. The change was from ebd661e3 to bf9ea5e4. Zellic conducted a diff audit between the previous scope and the new scope.

June 17th, 2026

A scope change occurred on June 17th, 2026. The change was from bf9ea5e4 to facaf223. Zellic conducted a diff audit between the previous scope and the new scope.

June 23rd, 2026

A scope change occurred on June 23rd, 2026. The scope was migrated to a public repository monorepo-external and the commit was updated from facaf223 to an equivalent commit in the new repository dcade8e7.

July 22nd, 2026

A scope change occurred on July 22nd, 2026. The change was from dcade8e7 ↗ to 0781f4e6 ↗. Zellic conducted a diff audit between the previous scope and the new scope.

5. System Design

As part of the audit process, Zellic performed a technical comparison between the Stellar implementation of the LayerZero OneSig, translated from the original Solidity/Move code, and its EVM counterpart. The goal was to identify vulnerabilities and bugs introduced during the translation as well as incorrect implementations that deviate from the protocol's specification.

This provides an overview of the differences between the Stellar and EVM implementations of the LayerZero OneSig.

OneSig design

OneSig is an open-source solution for executing any number of transactions across any number of blockchains with a single signature.

The core difference between the OneSig Stellar implementation from the reference EVM implementation arises due to the way authentication is handled in Stellar Soroban compared to the EVM.

In Stellar Soroban, authorization is not implicitly done via allowances/storage/delegates but instead carried out via authorization trees, in which every cross-contract call that requires authorization (e.g., a token transfer) is supplied by the account in the form of an auth entry.

Due the nature of OneSig being a contract account, this authorization is implemented by the `__check_auth` function as expected of contract accounts, which is invoked whenever `onesig.require_auth()` or any similar variants are called.

Another core difference may be the fact that the signers in `onesig-stellar` not only commit to the top-level calls when generating the Merkle proofs and their signatures, but they are also committing to each subinvocation generated by that root function call. This is different from the EVM reference where only top-level functions are committed to.

There is one ambiguous situation, which is unpreventable with this design, and that is subinvocations in the signed auth entries cannot be guaranteed to be run in the order in which they are signed — for example:

```
FOO.root_call
    - sub_invo.a()
    - sub_invo.b()
```

If there is some global storage that determines whether `root_call` executes `sub_invo.a()` or `sub_invo.b()` first, the signers cannot commit to any structure that will guarantee only one subset of execution; both executions will be considered valid against the signed auth tree. It is up to the signers to avoid potential scenarios by not committing to ambiguous smart contracts where execution paths can change via front-running or other issues.

6. Assessment Results

During our assessment on the scoped LayerZero Stellar OneSig files, we discovered three findings. No critical issues were found. One finding was of medium impact and two were of low impact.

6.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.